



SAPIENZA
UNIVERSITÀ DI ROMA

Centralized ILP Orchestrator for Task Routing in Satellite Networks

Faculty of Information Engineering, Computer Science and Statistics
Computer Science

Claudio Bagini

ID number 2045337

Advisor

Dr. Emiliano Casalicchio

Co-Advisor

Academic Year 2025/2026

Thesis not yet defended

Centralized ILP Orchestrator for Task Routing in Satellite Networks
Bachelor's Thesis. Sapienza University of Rome

© 2026 Claudio Bagini. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Version: August 14, 2026

Author's email: bagini.2045337@studenti.uniroma1.it

Dedication

Abstract

The evolution of Low Earth Orbit (LEO) satellite megaconstellations has opened new perspectives for Satellite Computing, enabling the extension of edge computing capabilities directly into space (Orbital Edge Computing). However, the highly dynamic network topology, stringent energy constraints of satellite nodes, and strict deadlines of real-time applications make task orchestration and offloading critical challenges.

This thesis presents the design, implementation, and performance evaluation of a centralized orchestrator based on Integer Linear Programming (ILP). To mitigate solver computational costs and prevent starvation, the system integrates a dynamic batching mechanism controlled by Batch Size (BS) and Batch Timeout (BT) metrics, along with a coordinated formulation of timing constraints that accounts for queuing delay prior to resolution.

Using discrete-event simulations developed with the SimPy framework, the architecture was evaluated under stress across varying request arrival rates (λ), comparing *Mapping Time* and *Mapping Energy* optimization strategies. A detailed analysis revealed that system collapse under heavy loads ($\lambda > 6$ req/s) is predominantly driven by timeout deadlines (*Deadline Exceeded*) rather than physical compute resource saturation. Furthermore, sensitivity experiments demonstrated the effectiveness of deadline relaxation (ΔD) in recovering high completion rates.

Finally, the centralized orchestrator was benchmarked against state-of-the-art distributed and heuristic solutions (*OrbitAware*, *DTS-base*, *DTS-APopt*, and distributed ILP). The results quantify the fundamental trade-off between the theoretical global optimality of centralized allocation and the impact of queuing latency compared to the responsiveness of distributed models.

Acknowledgments

Acknowledgments

Contents

1	Introduction	1
1.1	Motivation and Context	1
1.2	Problem Statement	2
1.3	Objectives and Main Contributions	3
1.4	Thesis Outline	4
2	Background and State of the Art	5
2.1	Satellite Computing Paradigms	5
2.2	Task Offloading and Routing in LEO Networks	6
2.2.1	System Entities and Network Topology	6
2.2.2	Workload Profiles and Constraints	7
2.3	Centralized vs. Distributed Orchestration	7
2.3.1	Centralized Orchestration	7
2.3.2	Distributed Orchestration	7
2.4	State-of-the-Art Approaches	8
2.4.1	Distributed Heuristic Frameworks	8
2.4.2	Optimization-Based and Distributed ILP Formulations	8
2.5	Research Gap and Thesis Positioning	9
3	System Architecture, Mathematical Formulation, and Implementation	10
3.1	Regional Constellation Architecture and Offloading Model	10
3.2	Event-Driven Dynamic Batching Engine	11
3.3	Constellation Topology Mapping and Metric Normalization	14
3.4	Centralized Integer Linear Programming Formulation	17
3.4.1	Latency, Energy, and Orbital Eclipse Modeling	17
3.4.2	Multi-Objective Function and System Constraints	18
3.4.3	Solver Routine and Cascade Rejection Diagnostics	18
3.5	Task Execution and Resilient Hybrid Routing	24
3.6	Computational Complexity and Scalability Analysis	27
3.6.1	Topology Mapping Complexity	27
3.6.2	ILP Formulation and Solver Complexity	27
3.6.3	Hybrid Routing Complexity	28

4	Performance Analysis and Evaluation	29
4.1	Simulation Framework	29
4.2	Experimental Setup and Workload Profiles	29
4.3	Evaluation of Optimization Strategies	29
4.3.1	Mapping Time vs. Mapping Energy Comparison	29
4.3.2	Impact of Batching Parameters (BS and BT)	29
4.4	Bottleneck and Collapse Analysis	29
4.4.1	System Behavior Under Heavy Load ($\lambda > 6$ req/s)	29
4.4.2	Physical Saturation vs. Deadline Exceeded Failures	29
4.5	Sensitivity Analysis: Deadline Relaxation (ΔD)	29
5	Comparative Benchmark	30
5.1	Experimental Setup and Baseline Algorithms	30
5.2	Comparative Performance Evaluation	30
5.3	Trade-Off Analysis	30
6	Conclusions and Future Work	31
6.1	Summary of Findings	31
6.2	Limitations of the Centralized Approach	31
6.3	Future Research Directions	31
	Bibliography	32

Chapter 1

Introduction

In recent years, space communications and satellite networks have undergone a monumental paradigm shift. The rapid miniaturization of space components, combined with drastically reduced launch costs, has catalyzed the deployment of Mega-Constellations operating in Low Earth Orbit (LEO). Traditionally, satellite architectures functioned strictly under a "bent-pipe" model, where satellites served as passive transparent relays forwarding raw telemetry or communication signals back to Ground Stations (GS) for processing. However, the unprecedented volume of data produced by modern spaceborne applications—such as high-resolution Earth Observation (EO), environmental monitoring, and ubiquitous Internet of Things (IoT) sensing—has rendered this traditional pipeline highly inefficient.

To overcome the severe bandwidth limits of space-to-ground downlinks and the high propagation delays associated with remote terrestrial processing, the paradigm of **Orbital Edge Computing (OEC)**—also broadly termed Satellite Computing (SC)—has emerged. By equipping LEO satellites with onboard processing hardware, data can be computed directly in space close to the source, transmitting only actionable insights or highly compressed results to ground users. Despite its immense potential, executing compute-intensive tasks on resource-constrained satellite nodes introduces complex orchestration challenges that require advanced decision-making frameworks.

1.1 Motivation and Context

The transition toward Orbital Edge Computing is driven by two main technological trends: the exponential increase in space-generated data and the strict temporal requirements of next-generation applications. Applications such as real-time disaster response, autonomous maritime navigation, and defense surveillance demand sub-second latency for task execution and reporting. Processing these tasks on Earth introduces unavoidable latency bottlenecks due to intermittent Ground Station contact windows, dynamic weather interference, and severe downlink congestion.

LEO megaconstellations (such as Starlink, OneWeb, and Kuiper) consist of hundreds to thousands of satellites orbiting at altitudes between 300 km and 2000 km. These satellites move at high orbital velocities (approximately 7.5 km/s relative to the Earth's surface), resulting in a continuously changing network topology. Inter-

Satellite Links (ISLs)—utilizing high-throughput Free-Space Optics (FSO) or Radio Frequency (RF) channels—enable direct communication between adjacent satellites without routing signals down to the ground.

Consequently, a LEO constellation effectively forms a distributed, high-velocity edge computing network in orbit. However, executing applications across this orbital fabric requires intelligent **Task Offloading and Routing Orchestration**. Ground users or source satellites generate computational tasks that must be dynamically assigned to suitable processing nodes across the constellation. Deciding *where* and *when* to execute a given task, while routing its associated payload across multiple dynamic ISL hops, constitutes a high-dimensional optimization problem that directly governs overall network performance.

1.2 Problem Statement

Orchestrating task offloading in Orbital Edge Computing environments presents a unique set of constraints and operational challenges that set it apart from conventional terrestrial cloud/edge systems:

- **Severe Energy Budget Limits:** Satellite Edge Nodes (SENs) rely on solar panels and onboard battery storage. Computationally intensive task execution and RF/optical transmissions consume substantial power. Over-utilizing specific nodes leads to rapid battery depletion, thermal overhead, or total operational failure.
- **Heterogeneous and Limited Hardware Resources:** Onboard processing units (e.g., embedded CPUs, specialized AI accelerators) are constrained by strict Space, Weight, and Power (SWaP) requirements, as well as radiation-hardening constraints.
- **Dynamic Topology and Intermittent Connectivity:** High orbital mobility leads to frequent topology reconfigurations, changing link distances, and periodic loss of line-of-sight. Task routing paths must be continuously recalculated to avoid dropped links.
- **Strict Temporal Constraints and Queuing Delays:** Tasks possess strict deadlines (D_r). If the sum of transmission latency, queuing delay, and computation time exceeds D_r , the task fails, resulting in wasted system energy and degraded quality of service.

To achieve theoretical optimality in task placement and energy management, mathematical optimization tools such as **Integer Linear Programming (ILP)** are commonly employed. An ILP formulation can search the joint solution space of task routing, CPU allocation, and energy minimization to find globally optimal schedules.

However, centralized ILP solvers suffer from inherent computational complexity (NP-hard). In dynamic satellite networks with continuously arriving workloads, solving an ILP instance for every individual task request introduces unacceptable

computational overhead and processing latency at the central orchestrator. Conversely, accumulating task requests to solve them in batches introduces a trade-off: larger batches allow the ILP solver to achieve superior global resource allocation, but prolonged waiting times in orchestrator queues increase the risk of task deadline expiration (*Deadline Exceeded* failures).

1.3 Objectives and Main Contributions

The main objective of this thesis is to design, implement, and rigorously evaluate a **Centralized ILP Orchestration Framework** optimized for task routing and resource allocation in LEO satellite networks, incorporating a **Dynamic Batching Mechanism** to balance solver computational complexity with network queuing delay.

Specifically, this work addresses the key question: *To what extent can a centralized optimization approach maintain global allocation efficiency in high-density satellite networks before queuing and solver overhead cause systemic performance degradation?*

The main contributions of this thesis are summarized as follows:

1. **Formalization of the Centralized ILP Optimization Model:** We formulate a comprehensive ILP mathematical model that jointly optimizes task routing across Inter-Satellite Links and computational resource allocation on Satellite Edge Nodes. The model supports dynamic weighting between latency optimization (*Mapping Time*) and total energy preservation (*Mapping Energy*).
2. **Design of an Integrated Dynamic Batching & Timeout Mechanism:** To mitigate the computational overhead of solving the ILP model continuously, we integrate an adaptive batching policy governed by Batch Size (*BS*) and Batch Timeout (*BT*). Furthermore, we propose a mathematical deadline compensation formula to adjust task deadlines based on orchestrator queuing duration (T_{batch}).
3. **Development of a Discrete-Event Simulation Environment:** We construct a custom, high-fidelity discrete-event simulation framework in Python using *SimPy*. The simulator accurately models request arrival dynamics (λ), physical network links, queue management, hardware processing, and energy consumption under varied workload conditions.
4. **In-Depth Performance and Bottleneck Analysis:** We perform extensive stress testing under high workload arrival rates ($\lambda > 6$ req/s) to identify system collapse thresholds, distinguishing between physical resource saturation (CPU/Network capacity) and temporal violations (*Deadline Exceeded*). We also conduct sensitivity analyses regarding deadline relaxation (ΔD).
5. **Comparative Benchmark Against State-of-the-Art Solutions:** We evaluate the centralized ILP orchestrator against established baseline heuristics (*OrbitAware*, *DTS-base*, *DTS-APopt*) and distributed ILP formulations from the literature, establishing clear operational trade-offs regarding task completion rate, overall latency, and energy efficiency.

1.4 Thesis Outline

The remainder of this thesis is structured as follows:

- **Chapter 2 (Background and State of the Art):** Reviews the fundamental concepts of Satellite Computing, LEO network dynamics, and existing literature on task offloading. It provides a taxonomy of centralized versus distributed orchestration schemes and details key reference algorithms.
- **Chapter 3 (System Architecture and ILP Formulation):** Presents the formal mathematical system model, decision variables, objective functions (*Mapping Time* vs. *Mapping Energy*), constraints, and the dynamic batching/timeout mechanisms.
- **Chapter 4 (Performance Analysis and Evaluation):** Describes the SimPy discrete-event experimental setup and evaluates the proposed centralized orchestrator under varying batching configurations, workload profiles, heavy-load conditions, and deadline relaxation scenarios.
- **Chapter 5 (Comparative Benchmark):** Details the comparative analysis benchmarking our centralized orchestrator against state-of-the-art distributed heuristics and formulations, highlighting key trade-offs in latencies, energy consumption, and global optimality.
- **Chapter 6 (Conclusions and Future Work):** Summarizes the main findings of this research, outlines limitations of centralized orbital orchestration, and proposes future research directions toward hybrid adaptive architectures.

Chapter 2

Background and State of the Art

The rapid proliferation of Low Earth Orbit (LEO) satellite constellations has fundamentally redefined the landscape of satellite communications and space-borne computing. Transitioning from traditional passive data relays to active processing nodes requires a comprehensive understanding of orbital dynamics, task offloading models, and orchestration strategies. This chapter provides the theoretical foundation and contextual literature for this thesis. It details the paradigms of Orbital Edge Computing (OEC), analyzes task offloading and routing dynamics in LEO networks, compares centralized and distributed orchestration models, reviews state-of-the-art baseline solutions, and articulates the research gap addressed by our work.

2.1 Satellite Computing Paradigms

Space-based networking architectures have historically been categorized by their orbital altitudes: Geostationary Earth Orbit (GEO), Medium Earth Orbit (MEO), and Low Earth Orbit (LEO). GEO satellites, positioned at approximately 35,786 km, offer stationary coverage over vast geographic areas but suffer from high round-trip propagation delays (exceeding 250 ms) and high free-space path loss. MEO satellites (2,000 km – 35,786 km) reduce latency slightly but remain unsuitable for ultra-low-latency processing.

In contrast, LEO satellites operate at altitudes ranging from 300 km to 2,000 km. Their proximity to Earth drastically lowers propagation latency (10 – 40 ms round-trip) and reduces power loss, making them ideal for high-throughput, real-time communications. However, LEO satellites travel at extreme velocities relative to the surface (~ 7.5 km/s), completing an orbit in approximately 90 – 120 minutes. This rapid mobility results in dynamic topologies where individual satellites remain in line-of-sight with a fixed ground position for only a few minutes.

To process data efficiently within this dynamic environment, satellite network architectures have evolved through three primary paradigms:

1. **Transparent Bent-Pipe Relay:** Satellites act merely as transparent repeaters. Raw data captured by Earth Observation (EO) sensors or transmitted by Ground Users (GUs) is amplified and forwarded directly to Ground Stations (GSs). Computation occurs entirely in terrestrial cloud centers. This

model suffers from severe downlink bottlenecks and complete connectivity loss when flying over remote regions (e.g., oceans or polar zones) lacking GS infrastructure.

2. **Terrestrial Cloud Offloading with Inter-Satellite Routing:** Satellites utilize Inter-Satellite Links (ISLs) to route raw data across orbital planes until a satellite with an active GS connection is reached. While this mitigates local GS coverage gaps, the volume of raw data still strains optical and RF link bandwidth.
3. **Orbital Edge Computing (OEC) / Satellite Edge Computing (SEC):** Satellites are equipped with onboard processing units—termed Satellite Edge Nodes (SENs)—capable of executing compute-intensive algorithm payloads (e.g., AI inference, image compression, signal filtering) directly in orbit. Raw data is processed locally at the source node or offloaded across ISLs to adjacent SENs. Only refined, actionable insights or small computation results are downlinked to Earth, drastically reducing bandwidth consumption and end-to-end response times.

2.2 Task Offloading and Routing in LEO Networks

Task offloading in Orbital Edge Computing involves receiving task execution requests generated by Ground Users (GUs) or space-borne sensors and strategically allocating their computational execution across available SENs while routing payloads through Inter-Satellite Links (ISLs).

2.2.1 System Entities and Network Topology

An OEC network architecture consists of three core structural entities:

- **Ground Users (GUs) / Remote Sensors:** Distributed terrestrial entities that issue computational task requests r_i . Each request is characterized by a data payload size D_r^{data} (in bits), required CPU compute cycles W_r , and a strict operational deadline D_r (in seconds).
- **Satellite Edge Nodes (SENs):** LEO satellites equipped with CPU cores, memory modules, and onboard energy reserves (solar panels and battery banks). Each SEN s_j possesses a processing capacity f_j (cycles/second) and an operational energy budget E_j .
- **Inter-Satellite Links (ISLs):** High-speed communication channels connecting adjacent satellites. Intra-plane ISLs connect satellites within the same orbital plane (stable distance), whereas inter-plane ISLs connect satellites in adjacent planes (dynamically varying distance and pointing angles). Each ISL link (u, v) is constrained by bandwidth B_{uv} and propagation delay τ_{uv} .

2.2.2 Workload Profiles and Constraints

Workloads submitted to an OEC network are typically heterogeneous, categorized into two main profiles:

- **Real-Time Workloads:** Tasks generated by time-critical applications (e.g., emergency disaster detection, defense alerts) requiring immediate processing with tight deadlines D_r .
- **Batch / Delay-Tolerant Workloads:** Tasks associated with routine telemetry, scientific processing, or background data aggregation with relaxed deadlines.

Offloading decisions must jointly satisfy physical compute constraints ($\sum W_r \leq f_j$), network bandwidth limits, energy consumption limits (computation power P_{comp} and transmission power P_{tx}), and strict end-to-end deadline bounds ($T_{total} = T_{propagation} + T_{transmission} + T_{queue} + T_{computation} \leq D_r$).

2.3 Centralized vs. Distributed Orchestration

Managing resource allocation across hundreds of moving SENs requires an orchestration paradigm. Literature categorizes these paradigms into two primary architectures: Centralized and Distributed.

2.3.1 Centralized Orchestration

In a centralized orchestration architecture, a dedicated master entity—such as a designated lead satellite (Ground/Orbital Orchestrator) or a ground control station—collects global network state information (CPU load, battery state, link traffic) from all SENs.

- **Advantages:** The central orchestrator maintains complete global visibility over all network resources, enabling mathematically optimal decision-making. Global optimization models (e.g., Integer Linear Programming) can eliminate load imbalances, prevent local hotspots, and minimize network-wide energy consumption.
- **Disadvantages:** Centralized schemes incur significant control signaling overhead to aggregate state information. Furthermore, requests must travel to the orchestrator, causing queuing delays and introducing a single point of failure and bottleneck under high request arrival rates (λ).

2.3.2 Distributed Orchestration

In distributed orchestration, individual SENs or localized clusters make autonomous offloading and routing decisions based solely on locally observed state information or partial neighborhood telemetry.

- **Advantages:** Distributed schemes eliminate centralized control bottlenecks, offer sub-millisecond decision latency, and demonstrate high fault tolerance against dynamic topology changes and satellite failures.
- **Disadvantages:** Lacking global visibility, distributed algorithms often converge to sub-optimal solutions. They are prone to localized network congestion, unbalanced CPU utilization, and premature battery depletion on popular forwarding paths.

2.4 State-of-the-Art Approaches

To place our research in context, we review key baseline algorithms and mathematical models established in recent IEEE literature on Orbital Edge Computing and Task Offloading.

2.4.1 Distributed Heuristic Frameworks

Heuristic approaches sacrifice theoretical optimality in favor of computational speed and low decision latency:

- **OrbitAware Architecture:** A dynamic heuristic framework designed specifically for highly dynamic LEO topologies. *OrbitAware* leverages periodic orbital trajectory updates to predict link availability and select shortest processing/routing paths locally, reducing control overhead at the cost of potential link queue saturation.
- **DTS-base (Dynamic Task Scheduling - Base):** A greedy distributed scheduling baseline that assigns incoming tasks to the nearest available SEN with sufficient idle CPU capacity. While fast, it ignores global energy budgets and multi-hop link traffic.
- **DTS-APopt (Dynamic Task Scheduling - Access Point Optimized):** An enhanced version of *DTS-base* that incorporates multi-criteria decision metrics, optimizing both the local access point selection and initial ISL hop bandwidth before dispatching tasks.

2.4.2 Optimization-Based and Distributed ILP Formulations

To improve allocation quality, recent studies have formulated mathematical optimization models using Integer Linear Programming (ILP) and mixed-integer techniques:

- **Distributed / Hierarchical ILP:** Frameworks that decompose a global ILP problem into smaller sub-problems solved locally across satellite clusters. While more scalable than pure centralized ILP, distributed ILP approaches suffer from convergence delays and coordination overhead during inter-cluster task transfers.

- **Weighted Objective Functions:** Optimization models in the literature frequently balance competing goals—such as latency minimization versus energy preservation—by assigning scalar weights (W_{lat}, W_{en}) to objective terms.

2.5 Research Gap and Thesis Positioning

While state-of-the-art distributed heuristics offer rapid processing times and distributed ILP frameworks attempt local decomposition, a significant research gap remains regarding the practical limits of **Centralized ILP Orchestration** in LEO satellite networks.

Existing literature frequently assumes either ideal zero-latency centralized orchestrators or dismisses centralized optimization entirely due to its computational complexity. Few works rigorously examine the trade-offs when a centralized ILP orchestrator is equipped with an integrated **Dynamic Batching** and **Timeout Mechanism** (BS/BT) to manage request queues under heavy workloads ($\lambda > 6$ req/s).

This thesis addresses this exact gap by:

1. Formalizing a centralized ILP model that dynamically toggles between latency minimization (*Mapping Time*) and energy conservation (*Mapping Energy*).
2. Quantifying the systemic performance bounds of centralized optimization when coupled with dynamic request batching and queuing delay compensation.
3. Conducting a rigorous comparative benchmark against established state-of-the-art heuristics (*OrbitAware*, *DTS-base*, *DTS-APopt*) and distributed ILP variants under realistic discrete-event simulation conditions (*SimPy*).

Chapter 3

System Architecture, Mathematical Formulation, and Implementation

This chapter provides a comprehensive dissertation on the theoretical framework, mathematical optimization formulation, and concrete software architecture of the proposed Centralized Integer Linear Programming (ILP) Orchestrator. To avoid conceptual disconnection between the theoretical modeling and its practical realization, each section progressively couples the formal system dynamics with its executable implementation in Python, utilizing the `SimPy` discrete-event simulation engine and the `PuLP` mathematical programming library backed by the `HiGHS` solver.

We formalize the regional Low Earth Orbit (LEO) constellation environment, derive the multi-objective optimization problem governing task assignment and multi-hop Inter-Satellite Link (ISL) routing, detail the dynamic batching and greedy recovery mechanisms, and conclude with a rigorous computational complexity analysis.

3.1 Regional Constellation Architecture and Offloading Model

We consider a regional orbital edge computing network deployed within a Low Earth Orbit (LEO) satellite constellation. The system comprises a set of candidate Satellite Edge Nodes (SENs), denoted by $S = \{1, 2, \dots, n\}$, and a set of regional Access Points (APs) or satellite gateways AP_{global} . Within the operational region, a designated *Master* functions as the centralized orchestrator, maintaining global visibility over constellation kinematics, dynamic processor queue occupancies, residual battery energy budgets, and active inter-satellite link topologies.

Ground Users (GUs) generate computational requests that arrive dynamically at the regional gateways. Each offloading request $r \in R$ is characterized by a five-element tuple:

$$r = (d_{cpu}, D_r, s_r, t_{arr}, AP_r) \quad (3.1)$$

where d_{cpu} is the computational CPU workload demand (in seconds), D_r represents

the maximum allowable service processing deadline, s_r denotes the input/output image size (in MB), t_{arr} is the generation timestamp, and $AP_r \in AP_{global}$ identifies the originating entry point.

To achieve robust real-time execution in highly dynamic space environments, the operational lifecycle of task r is modeled as a decoupled, three-phase circuit:

1. **Uplink Control and Forwarding Phase (T_{up}):** The originating gateway AP_r injects a lightweight control payload $S_{req} = 2$ KB into the orbital plane, which is routed across $h_{up}(r, i)$ ISL hops to candidate satellite $i \in S$.
2. **On-Board Computational Execution Phase (T_{cpu}):** Satellite i processes d_{cpu} units of work, incurring both the deterministic waiting time in the on-board processor queue $W_{cpu, i}$ and the physical execution delay.
3. **Post-Execution Geographic Result Delivery (T_{down}):** Upon computation completion, the generated output payload is routed back from the executing satellite i to the originating Access Point AP_r across the dynamic ISL topology via an autonomous geographic greedy routing protocol.

Because result return is governed dynamically in real time by the autonomous greedy protocol, the centralized ILP orchestrator focuses its optimization capacity on the joint satellite selection, uplink multi-hop routing, and processing deadline compliance.

3.2 Event-Driven Dynamic Batching Engine

In a high-throughput satellite environment, invoking an NP-hard ILP solver upon each individual task arrival introduces prohibitive computational overhead and CPU starvation. To address this, the Master Orchestrator implements an event-driven dynamic batching mechanism inside a `SimPy` discrete-event framework, governed by two competing thresholds:

- **Maximum Batch Size (BS):** A structural threshold (`centralized_batch_size`) that limits the maximum number of requests solved concurrently, bounding the solver variable space $|R| \times |S|$.
- **Batch Timeout (BT):** A temporal threshold (`centralized_batch_timeout`) that enforces a maximum buffering delay, ensuring requests are not stalled indefinitely.

The execution loop remains quiescent until the arrival of the first request into `task_buffer`, which immediately succeeds `item_arrived_event` and initiates the timeout timer. The batch is dispatched when either $|Q| \geq BS$ (triggering `batch_ready_event`) or when the elapsed time satisfies $\Delta t \geq BT$.

A critical design requirement in discrete-event simulation is ensuring state consistency. Because satellite positions, CPU queue lengths, and energy levels evolve continuously, the orchestrator executes a zero-delay yield (`yield self.env.timeout(0)`) immediately prior to batch processing. This yields thread execution back to the

SimPy event loop, ensuring that all physical updates across the constellation at timestamp t are fully committed before optimization begins.

Listing 3.1 presents the software architecture of the `Orchestrator` class initialization, request enqueueing, and the background lifecycle process.

```

1  import heapq
2  import simpy
3  import globals
4  import pulp
5  import ILP_simulation
6  import utils
7  from Task import Task
8  from EdgeServer import get_pos_proximity
9
10
11 class Orchestrator:
12     def __init__(self, env, config):
13         self.env = env
14         self.config = config
15
16         # Retrieve batch configuration parameters from config dictionary
17         self.batch_size = config.get("centralized_batch_size", 5)
18         self.batch_timeout = config.get("centralized_batch_timeout", 0.25)
19
20         self.task_buffer = []
21
22         # SimPy event handlers for dynamic batching triggers
23         self.item_arrived_event = self.env.event()
24         self.batch_ready_event = self.env.event()
25
26         # Spawn the orchestrator background process in SimPy
27         self.env.process(self._run_loop())
28
29     def add_task(self, task_data, task_id, visible_aps):
30         """
31         Enqueue an incoming task into the buffer and handle event triggers.
32         """
33         if not self.task_buffer and not self.item_arrived_event.triggered:
34             self.item_arrived_event.succeed()
35
36         if "arrival_time" not in task_data:
37             task_data["arrival_time"] = self.env.now
38
39         self.task_buffer.append({
40             "task_id": task_id,
41             "data": task_data,
42             "visible_aps": visible_aps,
43             "arrival_time": self.env.now
44         })
45
46         if len(self.task_buffer) >= self.batch_size and not self.batch_ready_event.triggered:
47             self.batch_ready_event.succeed()
48
49     def _run_loop(self):
50         """
51         Main lifecycle of the orchestrator, managing event-driven waits without blocking

```

```

52  ↪ the simulation environment.
53      """
54      while True:
55          # 1. Wait idle until the arrival of the FIRST task in the buffer
56          if not self.task_buffer:
57              yield self.item_arrived_event
58
59          # 2. Initialize batch timer (Timeout)
60          timeout_event = self.env.timeout(self.batch_timeout)
61
62          # 3. Wait until either batch capacity is reached OR the timer expires
63          yield self.batch_ready_event | timeout_event
64
65          # 4. Process accumulated tasks in the current batch
66          if self.task_buffer:
67              trigger_reason = "Maximum Capacity" if self.batch_ready_event.triggered
68          ↪ else "Timeout"
69              print(f"[{self.env.now:.3f}] Orchestrator: ILP trigger due to {
70              ↪ trigger_reason} ({len(self.task_buffer)} tasks).")
71
72              # --- SIMPY STATE SYNC: FORCE CONSTELLATION STATE UPDATE ---
73              # Yield control to the event loop to ensure physical satellite state
74          ↪ updates
75              # (CPU queues, energy levels, orbital positions) at timestamp t.
76              yield self.env.timeout(0)
77              # -----
78
79              self._process_batch()
80
81              # 5. Reset events for the subsequent batching cycle
82              self.item_arrived_event = self.env.event()
83              self.batch_ready_event = self.env.event()
84
85      def _process_batch(self):
86          """
87          Core routine managing constellation state extraction, ILP resolution, and task
88          ↪ dispatching.
89          """
90          # A. Extract dynamic constellation network topology
91          network_state = self._map_constellation()
92
93          # B. Invoke the PuLP ILP solver routine
94          assignments = self._solve_ilp(network_state, self.task_buffer)
95          print(f"[{self.env.now:.3f}] Orchestrator: ILP execution completed.")
96
97          # C. Execute/dispatch tasks based on optimization decision matrix
98          self._dispatch_tasks(assignments)
99
100         # Clear task buffer for the next incoming batch
101         self.task_buffer.clear()

```

Listing 3.1. Orchestrator Class Architecture and SimPy Event Loop.

3.3 Constellation Topology Mapping and Metric Normalization

Before formulating the optimization model, the Master Orchestrator must compute the optimal multi-hop ISL routing paths from every active ground Access Point to all candidate Satellite Edge Nodes. This is handled by the `_map_constellation` method using a Multi-Source Dijkstra path algorithm.

Evaluating dynamic satellite links with raw physical dimensions introduces dimensional bias (e.g., comparing milliseconds of propagation delay against microjoules of radio energy). To achieve uniform metric evaluation across heterogeneous links, each directed edge (u, v) with latency $\tau_{u,v}$ and bandwidth $B_{u,v}$ (in Bytes/s) is evaluated against a standardized reference payload $S_{ref} = 1$ MB (1024^2 bytes):

$$t_{link}(u, v) = \tau_{u,v} + \frac{S_{ref}}{B_{u,v}} \quad (3.2)$$

$$e_{link}(u, v) = P_{net} \cdot \frac{S_{ref}}{B_{u,v}} \quad (3.3)$$

where P_{net} is the radio transmission power (`Ptrasm`).

The edge metrics are normalized against theoretical upper bounds (T_{max} and E_{max}) derived from minimum network bandwidth ($B_{min} = 100$ KB/s) and maximum link latency ($\tau_{max} = 100$ ms):

$$\hat{t}_{link}(u, v) = \frac{t_{link}(u, v)}{T_{max}}, \quad \hat{e}_{link}(u, v) = \frac{e_{link}(u, v)}{E_{max}} \quad (3.4)$$

The composite dimensionless edge weight $c_{link}(u, v)$ is computed as a convex combination parameterized by weights w_R (`centralized_w_r`) and w_e (`centralized_w_e`):

$$c_{link}(u, v) = w_R \cdot \hat{t}_{link}(u, v) + w_e \cdot \hat{e}_{link}(u, v) \quad (3.5)$$

Concurrently, the routine queries all candidate nodes in `globals.edge_servers` to capture residual energy B_i , processor queue occupancy $W_{cpu,i}$ (`W_cpu_simpy`), network queue occupancy $W_{net,i}$ (`W_net_simpy`), processor clock speed $C_{sen,i}$, CPU energy coefficient κ_i (`e_coeff`), and orbital sunset timestamp $t_{sunset,i}$.

Listing 3.2 shows the executable implementation of the constellation mapping procedure.

```

1  def _map_constellation(self):
2      """
3      Map optimal routing paths from each Access Point (AP) to all candidate
4      Satellite Edge Nodes (SENs) using Dijkstra's algorithm.
5      Link costs are computed as a weighted combination of Delay and Energy.
6      """
7      # Retrieve objective cost function weights from configuration settings
8      w_e = self.config.get("centralized_w_e", 0.5)
9      w_R = self.config.get("centralized_w_r", 0.5)
10     p_trasm = self.config.get("Ptrasm", 1.0)
11
12     # Standardize link evaluation using a 1 MB reference payload (in bytes)
13     REF_DATA_BYTES = 1024 * 1024

```

```

14     network_state = {
15         "routing_table": {}, # Format: {ap_name: {sen_name: {'total_cost': X, '
16 ↪ total_time': Y, 'total_energy': Z, 'path': [...]}}}
17         "node_states": {} # Format: {sen_name: {'energy_residual': X, 'W_cpu_simp
18 ↪ y': Y, ...}}
19     }
20
21     # 1. Collect all topology nodes to facilitate network graph traversal
22     all_nodes = globals.global_access_point + globals.edge_servers
23     node_dict = {n.name: n for n in all_nodes}
24
25     # 2. Multi-Source Dijkstra: Explore network topology starting from each AP
26     for ap in globals.global_access_point:
27         network_state["routing_table"][ap.name] = {}
28
29         # Priority queue tuple structure:
30         # (total_cost, total_time, total_energy, current_node_name, path_so_far)
31         pq = [(0.0, 0.0, 0.0, ap.name, [ap.name])]
32
33         # Track minimum discovered cost to reach each node from this AP
34         min_costs = {ap.name: 0.0}
35
36         while pq:
37             current_cost, current_time, current_energy, current_node_name, path =
38 ↪ heapq.heappop(pq)
39
40             # Skip obsolete (sub-optimal) popped paths
41             if current_cost > min_costs.get(current_node_name, float('inf')):
42                 continue
43
44             # If current node is a Satellite Edge Node (SEN), store its optimal
45 ↪ routing path
46             current_node_obj = node_dict[current_node_name]
47             if current_node_obj in globals.edge_servers:
48                 network_state["routing_table"][ap.name][current_node_name] = {
49                     "total_cost": current_cost,
50                     "total_time": current_time,
51                     "total_energy": current_energy,
52                     "path": path
53                 }
54
55             # Explore physical network neighbors of the current node
56             for neighbor in current_node_obj.get_neighbors():
57
58                 if neighbor.name not in node_dict:
59                     continue
60
61                 # Extract physical communication link metrics
62                 latency = current_node_obj.get_latency(neighbor) or 0.0
63                 bw_MBps = current_node_obj.get_bandwidth(neighbor) or 0.0
64                 bw_Bps = bw_MBps * (1024**2)
65
66                 # Link interrupted if available bandwidth is zero or negative
67                 if bw_Bps <= 0:
68                     continue

```

```

67         # Set estimated bounds for network normalization
68         MIN_BW_BPS = 100000 # Example: 100 KB/s
69         MAX_LATENCY = 0.1 # Example: 100 ms
70
71         # Calculate theoretical upper bounds for reference payload
72         MAX_LINK_TIME = MAX_LATENCY + (REF_DATA_BYTES / MIN_BW_BPS)
73         MAX_LINK_ENERGY = current_node_obj.compute_routing_energy(
    ↪ REF_DATA_BYTES, bw_Bps, p_trasm)
74
75         # Compute latency and energy required for reference payload
    ↪ transmission
76         trans_time = REF_DATA_BYTES / bw_Bps
77         link_time = latency + trans_time
78         link_energy = current_node_obj.compute_routing_energy(REF_DATA_BYTES,
    ↪ bw_Bps, p_trasm)
79
80         # Min-Max Normalization
81         norm_time = link_time / MAX_LINK_TIME if MAX_LINK_TIME > 0 else 0.0
82         norm_energy = link_energy / MAX_LINK_ENERGY if MAX_LINK_ENERGY > 0
    ↪ else 0.0
83
84         # Dimensionless combined cost function for the link
85         link_cost = (w_R * norm_time) + (w_e * norm_energy)
86
87         new_cost = current_cost + link_cost
88         new_time = current_time + link_time
89         new_energy = current_energy + link_energy
90
91         # Update shortest path if a lower-cost route is found
92         if new_cost < min_costs.get(neighbor.name, float('inf')):
93             min_costs[neighbor.name] = new_cost
94             heapq.heappush(pq, (new_cost, new_time, new_energy, neighbor.name
    ↪ , path + [neighbor.name]))
95
96     # 3. Map health and state metrics of candidate Satellite Edge Nodes (SENs)
97     for node in globals.edge_servers:
98         try:
99             w_cpu_simpy = node.W_cpu()
100         except Exception:
101             w_cpu_simpy = getattr(node, "W_cpu", lambda: 0.0)()
102
103         try:
104             w_net_simpy = node.W_net()
105         except Exception:
106             w_net_simpy = getattr(node, "W_net", lambda: 0.0)()
107
108         # -----
109         # Compute orbital sunset time
110         raw_sunset = getattr(node, "orbitalSunset", None)
111         if raw_sunset is not None:
112             safe_sunset = float(raw_sunset)
113         else:
114             safe_sunset = 0.0
115         # -----
116
117         # Construct node state snapshot
118         network_state["node_states"][node.name] = {

```

```

119         "energy_residual": node.energy,
120         "W_cpu_simpy": w_cpu_simpy,
121         "W_net_simpy": w_net_simpy,
122         "C_sen": getattr(node, "C_sen", 1e9),
123         "e_coeff": self.config.get("energy_coefficient", 5e-26),
124         "sunset_time": safe_sunset
125     }
126
127     return network_state

```

Listing 3.2. Constellation Topology and SEN State Mapping via Multi-Source Dijkstra.

3.4 Centralized Integer Linear Programming Formulation

Let R be the set of tasks extracted in the active batch and S the set of candidate Satellite Edge Nodes. The optimization problem defines two families of binary decision variables:

$$x_{r,i} \in \{0,1\}, \quad \forall r \in R, \forall i \in S \quad (1 \text{ if task } r \text{ is allocated to satellite } i; 0 \text{ otherwise}) \quad (3.6)$$

$$y_r \in \{0,1\}, \quad \forall r \in R \quad (1 \text{ if task } r \text{ is explicitly rejected; 0 otherwise}) \quad (3.7)$$

3.4.1 Latency, Energy, and Orbital Eclipse Modeling

For each candidate allocation (r, i) , the end-to-end processing delay $R_{r,i}$ evaluated by the solver accounts for multi-hop uplink ISL transmission across path $\vec{P}_{up}(AP_r, i)$ and local computing delay:

$$R_{r,i} = \underbrace{\sum_{(u,v) \in \vec{P}_{up}} \left(W_{net,u} + \frac{S_{req}}{B_{u,v}} + \tau_{u,v} \right)}_{T_{up}(r,i)} + (W_{cpu,i} + d_{cpu,r}) \quad (3.8)$$

The cumulative fleet energy expenditure $\epsilon_{r,i}$ evaluates the network forwarding energy across all intermediate ISL nodes along $\vec{P}_{up}$ alongside local CPU execution:

$$\epsilon_{r,i} = \sum_{(u,v) \in \vec{P}_{up}} P_{net} \frac{S_{req}}{B_{u,v}} + \kappa_i \cdot d_{cpu,r} \cdot C_{sen,i}^3 \quad (3.9)$$

To avoid assigning tasks to satellites nearing orbital eclipse, the orchestrator defines an orbital sunset penalty $\Omega_i \in [0, 1]$:

$$\Omega_i = \begin{cases} 1.0 - \frac{t_{sunset,i}}{\max_{k \in S}(t_{sunset,k})}, & \text{if } \max_{k \in S}(t_{sunset,k}) > 0 \\ 0.0, & \text{otherwise} \end{cases} \quad (3.10)$$

Satellites currently unreachable due to orbital eclipse ($t_{sunset,i} = 0$) receive the maximum penalty $\Omega_i = 1.0$.

3.4.2 Multi-Objective Function and System Constraints

Let $P_{drop} = 10^6$ represent a massive penalty scalar ensuring that task rejection is strictly penalized unless physical constraints are violated. The aggregated fleet latency (Z_{time}), energy (Z_{energy}), and eclipse penalty (Z_{sunset}) are defined as:

$$Z_{time} = \sum_{r \in R} \sum_{i \in S} x_{r,i} R_{r,i} + \sum_{r \in R} y_r P_{drop} \quad (3.11)$$

$$Z_{energy} = \sum_{r \in R} \sum_{i \in S} x_{r,i} \epsilon_{r,i} + \sum_{r \in R} y_r P_{drop} \quad (3.12)$$

$$Z_{sunset} = \sum_{r \in R} \sum_{i \in S} x_{r,i} \Omega_i \quad (3.13)$$

Depending on the configuration parameter `centralized_primary_objective` (Time vs. Energy), the objective function Z applies hierarchical weights ($\alpha = 10.0, \beta = 1.0, \gamma = 5.0$):

$$\min Z = \begin{cases} \alpha Z_{time} + \beta Z_{energy} + \gamma Z_{sunset}, & \text{if primary strategy is Time} \\ \alpha Z_{energy} + \beta Z_{time} + \gamma Z_{sunset}, & \text{if primary strategy is Energy} \end{cases} \quad (3.14)$$

The optimization space is constrained by four strict linear relations:

1. **Decision Uniqueness:** Every task must be assigned to exactly one satellite or dropped:

$$\sum_{i \in S} x_{r,i} + y_r = 1, \quad \forall r \in R \quad (3.15)$$

2. **Strict Service Processing Deadline:** Enforces that uplink forwarding and local execution do not exceed the deadline D_r :

$$x_{r,i} \cdot R_{r,i} \leq D_r, \quad \forall r \in R, \forall i \in S \quad (3.16)$$

3. **Distributed Fleet Energy Budget:** Enforces that the total energy allocated to satellite node k (across all tasks where k is an intermediate ISL forwarder or execution node) remains within its residual battery capacity B_k :

$$\sum_{r \in R} \sum_{i \in S} x_{r,i} \cdot \epsilon_{r,i,k} \leq B_k, \quad \forall k \in S \quad (3.17)$$

4. **Constellation Load Balancing (Anti-Congestion):** Prevents processor queue bottlenecks by bounding the maximum simultaneous task allocations per node:

$$\sum_{r \in R} x_{r,i} \leq \left\lfloor \frac{|R|}{|S|} \right\rfloor + 5, \quad \forall i \in S \quad (3.18)$$

3.4.3 Solver Routine and Cascade Rejection Diagnostics

The model is constructed in Python using PuLP and executed via the HiGHS solver engine. Upon resolution, if a task is rejected ($y_r = 1$), the orchestrator executes a cascade diagnostic evaluation across all reachable SENs to classify the primary drop reason:

- "No_Route": No viable ISL path exists from AP_r to any satellite.
- "Deadline_Violation": Predicted response time $R_{r,i}$ exceeds D_r .
- "Sunset_Violation": Response time exceeds available solar visibility horizon $t_{\text{sunset},i}$.
- "Energy_Exhaustion": Residual battery B_k is insufficient for multi-hop transmission or computing.
- "Load_Balancing_Rejected": Satellite reached its maximum capacity threshold.
- "Global_ILP_Conflict": Task rejected to preserve global objective optimality under shared fleet constraints.

Listing 3.3 presents the complete `_solve_ilp` implementation.

```

1  def _solve_ilp(self, network_state, tasks):
2      """
3      Open-Source Hierarchical PuLP Optimization Model
4      """
5      # Instantiate minimization optimization problem
6      m = pulp.LpProblem("SECMotionModel_Centralized_ILP", pulp.LpMinimize)
7
8      # Configuration parameters
9      primary_obj = self.config.get("centralized_primary_objective", "time")
10
11     # Fixed control payload for uplink request (2 KB)
12     S_REQ_BYTES = 2048
13     p_net = self.config.get("Ptrasm", 1.0)
14
15     R_set = tasks
16     S_set = list(network_state["node_states"].keys())
17     all_nodes = globals.global_access_point + globals.edge_servers
18     node_dict = {n.name: n for n in all_nodes}
19
20     # Decision variable definitions:  $x[r, i]$  in  $\{0, 1\}$ ,  $y[r]$  in  $\{0, 1\}$ 
21     x = pulp.LpVariable.dicts("x", ((r, i) for r in range(len(R_set)) for i in S_set)
    ↪ , cat='Binary')
22     y = pulp.LpVariable.dicts("y", (r for r in range(len(R_set))), cat='Binary')
23
24     # --- CONSTRAINT 1: Assignment or Drop Uniqueness ---
25     for r_idx in range(len(R_set)):
26         # Sum of satellite assignments plus rejection variable must equal 1
27         m += pulp.lpSum(x[r_idx, i] for i in S_set) + y[r_idx] == 1, f"
    ↪ Assign_Or_Drop_{r_idx}"
28
29     time_costs = {}
30     energy_costs = {}
31
32     node_energy_expressions = {k: [] for k in S_set}
33
34     # 2. Parameter construction and physical constraints
35     for r_idx, task in enumerate(R_set):
36         task_data = task["data"]
37         ap_origin = task["visible_aps"][0].name

```

```

38     d_cpu = task_data.get('d_cpu', 0.0)
39     deadline = task_data.get('deadline', 300.0)
40     s_r_bytes = task_data.get('image_size', 0.0) * 1024 * 1024
41
42     for i in S_set:
43         route_info_up = network_state["routing_table"].get(ap_origin, {}).get(i)
44
45         if not route_info_up:
46             m += x[r_idx, i] == 0, f"NoRoute_{r_idx}_{i}"
47             time_costs[r_idx, i] = 0
48             energy_costs[r_idx, i] = 0
49             continue
50
51         path_up = route_info_up["path"]
52         num_hops_up = len(path_up) - 1
53
54         t_up = 0.0
55         e_fwd_total = 0.0
56
57         # --- UPLINK Latency and Energy Computation ---
58         for h in range(num_hops_up):
59             node_curr = node_dict[path_up[h]]
60             node_next = node_dict[path_up[h+1]]
61             lat = node_curr.get_latency(node_next) or 0.0
62
63             bw_raw = node_curr.get_bandwidth(node_next) or 0.0
64             b_isl = (bw_raw / 8.0) * (1024 ** 2) if bw_raw > 0 else 0.0
65
66             if b_isl > 0:
67                 Wn = network_state["node_states"].get(node_curr.name, {}).get("
↪ W_net_simpy", 0.0)
68                 d_net_hop = S_REQ_BYTES / b_isl
69                 t_up += Wn + d_net_hop + lat
70
71                 if hasattr(node_curr, 'compute_routing_energy'):
72                     e_hop_fwd = node_curr.compute_routing_energy(S_REQ_BYTES,
↪ b_isl, p_net)
73                 else:
74                     e_hop_fwd = p_net * (S_REQ_BYTES / b_isl)
75
76                 e_fwd_total += e_hop_fwd
77
78                 if node_curr.name in S_set:
79                     node_energy_expressions[node_curr.name].append(x[r_idx, i] *
↪ e_hop_fwd)
80
81         # --- LOCAL CPU EXECUTION (Latency and Energy) ---
82         node_obj = node_dict[i]
83         c_sen_i = network_state["node_states"][i]["C_sen"]
84         e_coeff_i = network_state["node_states"][i]["e_coeff"]
85
86         # Execution latency
87         Wc = network_state["node_states"][i].get("W_cpu_simpy", 0.0)
88         t_cpu_local = Wc + d_cpu
89
90         # Execution energy
91         if hasattr(node_obj, 'compute_execution_energy'):

```

```

92         e_cpu_local = node_obj.compute_execution_energy(d_cpu, c_sen_i, e=
↪ e_coeff_i)
93     else:
94         e_cpu_local = d_cpu * e_coeff_i * (c_sen_i ** 3)
95
96     node_energy_expressions[i].append(x[r_idx, i] * e_cpu_local)
97
98     # --- METRIC AGGREGATION AND DEADLINE CONSTRAINT ---
99     time_costs[r_idx, i] = t_up + t_cpu_local
100     energy_costs[r_idx, i] = e_fwd_total + e_cpu_local
101
102     # Constraint 2: Strict Service Deadline Compliance
103     R_ri_for_constraint = t_up + t_cpu_local
104     m += x[r_idx, i] * R_ri_for_constraint <= deadline, f"Deadline_{r_idx}_{i}
↪ }"
105
106     # Constraint 3: Distributed Fleet Energy Budget Limit
107     for k in S_set:
108         budget_k = max(0.0, network_state["node_states"][k]["energy_residual"])
109         if node_energy_expressions[k]:
110             m += pulp.lpSum(node_energy_expressions[k]) <= budget_k, f"Energy_Limit_{
↪ k}"
111
112     # --- Constraint 4: Anti-Congestion and Load Balancing ---
113     num_satellites = len(S_set)
114     fair_share = len(R_set) // num_satellites if num_satellites > 0 else len(R_set)
115     MAX_TASKS_PER_NODE = fair_share + 5
116
117     for i in S_set:
118         m += pulp.lpSum(x[r_idx, i] for r_idx in range(len(R_set))) <=
↪ MAX_TASKS_PER_NODE, f"Max_Capacity_{i}"
119
120     # --- ORBITAL SUNSET PENALTY PREPARATION ---
121     valid_sunsets = [network_state["node_states"][k].get("sunset_time", 0.0) for k in
↪ S_set]
122     max_sunset = max(valid_sunsets + [1.0])
123
124     sunset_penalties = {}
125     for i in S_set:
126         s_time = network_state["node_states"][i].get("sunset_time", 0.0)
127         sunset_penalties[i] = 1.0 - (s_time / max_sunset) if max_sunset > 0 else 0.0
128
129     # 3. Hierarchical Optimization Formulation
130     PENALTY_VALUE = 1000000.0
131
132     obj_time = pulp.lpSum(x[r_idx, i] * time_costs[r_idx, i] for r_idx in range(len(
↪ R_set)) for i in S_set) + \
133         pulp.lpSum(y[r_idx] * PENALTY_VALUE for r_idx in range(len(R_set)))
134
135     obj_energy = pulp.lpSum(x[r_idx, i] * energy_costs[r_idx, i] for r_idx in range(
↪ len(R_set)) for i in S_set) + \
136         pulp.lpSum(y[r_idx] * PENALTY_VALUE for r_idx in range(len(R_set)))
137
138     obj_sunset = pulp.lpSum(x[r_idx, i] * sunset_penalties[i] for r_idx in range(len(
↪ R_set)) for i in S_set)
139
140     WEIGHT_PRIMARY = 10.0

```

```

141     WEIGHT_SECONDARY = 1.0
142     WEIGHT_SUNSET = 5.0
143
144     if primary_obj == "time":
145         m += WEIGHT_PRIMARY * obj_time + WEIGHT_SECONDARY * obj_energy +
↪ WEIGHT_SUNSET * obj_sunset, "Total_Objective"
146     else:
147         m += WEIGHT_PRIMARY * obj_energy + WEIGHT_SECONDARY * obj_time +
↪ WEIGHT_SUNSET * obj_sunset, "Total_Objective"
148
149     # 4. Problem Resolution via HiGHS Solver
150     solver = pulp.getSolver('HiGHS', msg=False)
151     m.solve(solver)
152
153     status = pulp.LpStatus[m.status]
154
155     assigned_energy_per_node = {i: 0.0 for i in S_set}
156     assigned_tasks_per_node = {i: 0 for i in S_set}
157
158     if status in ('Optimal', 'Suboptimal'):
159         for r_idx in range(len(R_set)):
160             if pulp.value(y[r_idx]) is not None and pulp.value(y[r_idx]) < 0.5:
161                 for i in S_set:
162                     if pulp.value(x[r_idx, i]) is not None and pulp.value(x[r_idx, i]
↪ ]) > 0.5:
163                         assigned_energy_per_node[i] += energy_costs.get((r_idx, i),
↪ 0.0)
164                         assigned_tasks_per_node[i] += 1
165
166     # Decision Matrix Parsing and Cascade Rejection Diagnostics
167     assignments = []
168     if status in ('Optimal', 'Suboptimal'):
169         for r_idx, task in enumerate(R_set):
170             if pulp.value(y[r_idx]) and pulp.value(y[r_idx]) > 0.5:
171                 ap_origin = task["visible_aps"][0].name
172                 deadline = task["data"].get('deadline', 300.0)
173                 specific_reason = "Unknown"
174
175                 reachable_sens = [i for i in S_set if network_state["routing_table"].
↪ get(ap_origin, {}).get(i)]
176
177                 if not reachable_sens:
178                     specific_reason = "No_Route"
179                 else:
180                     reasons_for_rejection = []
181                     for i in reachable_sens:
182                         R_ri = time_costs.get((r_idx, i), 0)
183                         E_ri = energy_costs.get((r_idx, i), 0)
184                         s_time_i = network_state["node_states"][i].get("sunset_time",
↪ 0)
185                         actual_energy_left = network_state["node_states"][i]["
↪ energy_residual"] - assigned_energy_per_node.get(i, 0)
186
187                         if R_ri == 0 or R_ri > deadline:
188                             reasons_for_rejection.append("Deadline_Violation")
189                         elif R_ri > s_time_i:
190                             reasons_for_rejection.append("Sunset_Violation")

```

```

191         elif actual_energy_left < E_ri:
192             reasons_for_rejection.append("Energy_Exhaustion")
193         elif assigned_tasks_per_node.get(i, 0) >= MAX_TASKS_PER_NODE:
194             reasons_for_rejection.append("Load_Balancing_Rejected")
195         else:
196             reasons_for_rejection.append("Global_ILP_Conflict")
197
198         if reasons_for_rejection:
199             specific_reason = max(set(reasons_for_rejection), key=
↪ reasons_for_rejection.count)
200         else:
201             specific_reason = "Unknown"
202
203         print(f"[{self.env.now:.3f}] Orchestrator: Task {task['task_id']}
↪ rejected by ILP. Cause: {specific_reason}")
204
205         assignments.append({
206             "task_id": task["task_id"], "data": task["data"],
207             "assigned_sen": None, "routing_path_up": [], "routing_path_down":
↪ [],
208             "status": f"Scartato_{specific_reason}"
209         })
210         continue
211
212         assigned_node = None
213         for i in S_set:
214             if pulp.value(x[r_idx, i]) and pulp.value(x[r_idx, i]) > 0.5:
215                 assigned_node = i
216                 break
217
218         if assigned_node:
219             ap_origin = task["visible_aps"][0].name
220             path_up = network_state["routing_table"][ap_origin][assigned_node][
↪ path"]
221             path_down = [assigned_node, ap_origin]
222
223             assignments.append({
224                 "task_id": task["task_id"], "data": task["data"],
225                 "assigned_sen": assigned_node, "routing_path_up": path_up,
226                 "routing_path_down": path_down, "status": "Assigned"
227             })
228         else:
229             print(f"[{self.env.now:.2f}] [ILP Centralized] Batch Infeasible. Status: {
↪ status}")
230             for task in R_set:
231                 assignments.append({
232                     "task_id": task["task_id"], "data": task["data"],
233                     "assigned_sen": None, "routing_path_up": [], "routing_path_down": [],
234                     "status": "Infeasible"
235                 })
236
237         return assignments

```

Listing 3.3. PuLP ILP Formulation, HiGHS Resolution, and Cascade Rejection Diagnostics.

3.5 Task Execution and Resilient Hybrid Routing

Once the ILP optimizer produces the assignment decision matrix, the orchestrator invokes `_dispatch_tasks` to spawn independent `_route_and_execute_task` processes for each accepted request. Due to high orbital velocities in LEO constellations, planned ISL links may experience unexpected line-of-sight obstruction or degradation mid-flight. To guarantee fault tolerance, packet routing employs a dynamic hybrid routing protocol:

- **Nominal Planned Routing (Phase A):** Data packets strictly follow the shortest multi-hop ISL trajectory $\vec{P}_{up}$ pre-computed during constellation mapping.
- **Emergency Geographic Greedy Fallback (Phase B):** If an intermediate link fails ($B_{u,v} \leq 0$), the current node autonomously triggers greedy geographic recovery. The node inspects all available physical neighbors, calculates their Euclidean distance to the destination satellite via `get_pos_proximity`, and forwards the packet to the closest non-visited node. Visited sets (`routing_task.visited`) prevent forwarding loops.

Upon packet arrival at the destination Satellite Edge Node, the transmission duration and hop count are logged in `globals.gbl_generated_tasks_data`, and the process yields to the discrete-event CPU simulator (`ILP_simulation.TaskAssignment_ILP`). Finally, task status and timing metrics are synchronized in the global dataset.

Listing 3.4 shows the hybrid routing and execution routine.

```

1  def _route_and_execute_task(self, assignment, all_nodes, dispatch_time):
2      task_id = assignment["task_id"]
3      data = assignment["data"]
4      sen_name = assignment["assigned_sen"]
5      sen_obj = all_nodes.get(sen_name)
6      path_up = assignment.get("routing_path_up", [])
7
8      ap_origin = path_up[0] if path_up else globals.global_access_point[0].name
9
10     # --- 1. TRANSIENT ROUTING TASK CREATION ---
11     routing_task = Task(
12         task_id=task_id,
13         current_node=ap_origin,
14         dest_node=sen_name,
15         routingInitTime=dispatch_time,
16         task_type=data["type"],
17         image_size=data["image_size"]
18     )
19     routing_task.weight = data["image_size"]
20     routing_task.visited.add(ap_origin)
21
22     start_node = all_nodes.get(ap_origin)
23     if start_node and routing_task not in start_node.tasks:
24         start_node.tasks.append(routing_task)
25
26     # --- 2. DYNAMIC NAVIGATION LOOP (Hybrid Routing: ILP -> GREEDY) ---
27     current_node_name = ap_origin
28     target_dest_name = sen_name

```

```

29     path_index = 0
30     is_greedy = False
31
32     while current_node_name != target_dest_name:
33         current_node_obj = all_nodes.get(current_node_name)
34
35         if not is_greedy:
36             # =====
37             # PHASE A: PLANNED ROUTE NAVIGATION (ILP)
38             # =====
39             next_planned = path_up[path_index + 1] if path_index < len(path_up) -
↪ 1 else None
40             next_planned_obj = all_nodes.get(next_planned) if next_planned else
↪ None
41
42             if next_planned_obj and current_node_obj.get_bandwidth(
↪ next_planned_obj) > 0:
43                 # Active link: follow Orchestrator plan
44                 yield self.env.process(
45                     utils.sendTask(self.env, routing_task, current_node_obj,
↪ next_planned_obj, "ILP_Centralized")
46                 )
47                 path_index += 1
48                 routing_task.visited.add(next_planned)
49             else:
50                 # BROKEN LINK: Trigger greedy recovery fallback!
51                 print(f"[{self.env.now:.3f}] Orchestrator: Interrupted link at
↪ node {current_node_name} (Task {task_id}). Activating GREEDY Fallback!")
52                 is_greedy = True
53
54         if is_greedy:
55             # =====
56             # PHASE B: EMERGENCY RECOVERY NAVIGATION (GREEDY)
57             # =====
58             target_dest_obj = all_nodes.get(target_dest_name)
59             dest_pos = target_dest_obj.getPositionVector(globals.list_in_conf)
60             ranker_neighbors = []
61             neighbors_list = getattr(current_node_obj, 'neighbors',
↪ current_node_obj.get_neighbors())
62
63             for server in neighbors_list:
64                 if current_node_obj.get_bandwidth(server) > 0:
65                     neighbor_distance = get_pos_proximity(dest_pos, server.
↪ getPositionVector(globals.list_in_conf))
66                     is_ap = getattr(server, 'is_acc_point', False)
67                     ranker_neighbors.append((server, neighbor_distance, is_ap))
68
69             ranker_neighbors.sort(key=lambda x: (not x[2], x[1]))
70
71             best_server = None
72             for neighbor_tuple in ranker_neighbors:
73                 # Prevent ping-pong loops between adjacent satellites
74                 if neighbor_tuple[0].name not in routing_task.visited:
75                     best_server = neighbor_tuple[0]
76                     break
77
78             if best_server:

```

```

79         yield self.env.process(
80             utils.sendTask(self.env, routing_task, current_node_obj,
↪ best_server, 'GREEDY_FALLBACK')
81         )
82         routing_task.visited.add(best_server.name)
83     else:
84         print(f"[{self.env.now:.3f}] Orchestrator: GREEDY Fallback failed
↪ (dead-end) at node {current_node_name}. Task {task_id} dropped.")
85         routing_task.routingEndTime = self.env.now
86         routing_task.label = "DROPPED_GREEDY_DEADEND"
87         globals.gbl_tasks.append(routing_task)
88         return
89
90     # =====
91     # PHASE C: COMMON SAFETY CONTROL
92     # =====
93     if getattr(routing_task, 'label', '') == 'TTL_EXPIRED':
94         print(f"[{self.env.now:.3f}] Orchestrator: Task {task_id} dropped due
↪ to expired TTL during routing.")
95         globals.gbl_tasks.append(routing_task)
96         return
97
98     current_node_name = routing_task.current_node
99
100    # --- 3. FINAL DESTINATION ARRIVAL ---
101    transfer_time_up = self.env.now - data.get("arrival_time", dispatch_time)
102    final_hops = routing_task.hop
103    init_time = routing_task.routingInitTime
104    end_time = self.env.now
105    algos = routing_task.algorithms_used
106
107    print(f"[{self.env.now:.3f}] Orchestrator: Task {task_id} arrived at {
↪ sen_name} in {transfer_time_up:.3f}s (Hops: {final_hops})")
108
109    # --- 4. METRICS & DATASET LOGGING ---
110    globals.initial_server_counter[sen_name] = globals.initial_server_counter.get
↪ (sen_name, 0) + 1
111    globals.gbl_task_hops[str(task_id)] = final_hops
112    globals.gbl_task_final_hops[str(task_id)] = final_hops
113    globals.gbl_generated_tasks_data.append({
114        "task_id": task_id, "arrival_time": data.get("arrival_time",
↪ dispatch_time), "type": data["type"],
115        "ram": data.get("required_ram", 0), "disk": data.get("required_disk", 0),
116        "image_size": data["image_size"], "exec_time": data["d_cpu"],
117        "transfer_time": transfer_time_up, "performed_transfers": final_hops,
118        "final_hops": final_hops, "execution_server": sen_name
119    })
120
121    # --- 5. DISCRETE-EVENT CPU SIMULATION EXECUTION ---
122    yield self.env.process(
123        ILP_simulation.TaskAssignment_ILP(
124            env=self.env, selected_server=sen_obj, task_id=task_id, image_size=
↪ data["image_size"],
125            arrival_time_system=data.get("arrival_time", dispatch_time), num_hops
↪ =final_hops,
126            transfer_time=transfer_time_up, task_type=data["type"], d_cpu=data["
↪ d_cpu"], D_r=data["deadline"],

```

```

127         net_bw_override_Bps=None, result_sink={}, allow_retry=False,
128         routing_already_charged=True, routing_energy_from=None,
↪ 129     transfer_already_simulated=True
130     )
131 )
132
133 # --- 6. POST-EXECUTION METRICS SYNCHRONIZATION ---
134 final_task = next((t for t in globals.gbl_tasks if t.id == task_id), None)
135 if final_task:
136     final_task.hop = final_hops
137     final_task.routingInitTime = init_time
138     final_task.routingEndTime = end_time
139     final_task.algorithms_used = algos
140 else:
141     routing_task.routingEndTime = end_time
142     routing_task.label = "REJECTED_POST_ROUTING"
143     globals.gbl_tasks.append(routing_task)

```

Listing 3.4. Task Execution and Dynamic Hybrid Routing Procedure.

3.6 Computational Complexity and Scalability Analysis

To understand the computational trade-offs of centralized ILP orchestration compared to distributed heuristics, we formally analyze the algorithmic time and space complexity across all pipeline stages.

3.6.1 Topology Mapping Complexity

The constellation mapping routine executes Dijkstra's algorithm from each active Access Point $|AP|$ across the constellation graph $G = (V, E)$, where $V = AP_{global} \cup S$ and E is the set of active ISL and ground links. Using a binary min-heap priority queue, the asymptotic time complexity is:

$$\mathcal{O}(|AP| \cdot (|E| + |V| \log |V|)) \quad (3.19)$$

Because satellite node degree in regular grid LEO constellations (e.g., Walker delta) is bounded by physical ISL transceivers ($|E| \leq 4|S|$), the mapping stage scales linearly with constellation size $|S|$.

3.6.2 ILP Formulation and Solver Complexity

For a batch containing $|R|$ requests and $|S|$ candidate Satellite Edge Nodes:

- **Decision Variable Space:** The model instantiates $|R| \cdot |S|$ binary assignment variables $x_{r,i}$ and $|R|$ rejection variables y_r , yielding a total binary dimension of $\mathcal{O}(|R| \cdot |S|)$.
- **Constraint Space:** The formulation constructs $|R|$ uniqueness constraints, $|R| \cdot |S|$ deadline constraints, $|S|$ fleet battery constraints, and $|S|$ load-balancing constraints, yielding an exact row complexity of:

$$\mathcal{O}(|R| \cdot |S| + |R| + 2|S|) \quad (3.20)$$

While solving general 0-1 Integer Linear Programs is NP-hard, with a worst-case time complexity of $\mathcal{O}(2^{|R| \cdot |S|})$, the branch-and-cut algorithm in **HiGHS** solves the linear relaxation at each search node in polynomial time. The dynamic batching threshold $BS < 20$ guarantees that the active problem space remains strictly bounded, maintaining mean solving times below 10 ms in practical regional operations.

3.6.3 Hybrid Routing Complexity

Under nominal operation, planned packet forwarding requires $\mathcal{O}(h_{up})$ pointer lookups along pre-computed paths. In the event of link failure, the greedy fallback evaluates the spatial distance of adjacent neighbors (k) in $\mathcal{O}(k \log k)$ time per hop. The visited set lookup operates in $\mathcal{O}(1)$ average time using hash tables, guaranteeing that emergency routing overhead remains negligible relative to optical ISL propagation delays.

Chapter 4

Performance Analysis and Evaluation

4.1 Simulation Framework

4.2 Experimental Setup and Workload Profiles

4.3 Evaluation of Optimization Strategies

4.3.1 Mapping Time vs. Mapping Energy Comparison

4.3.2 Impact of Batching Parameters (BS and BT)

4.4 Bottleneck and Collapse Analysis

4.4.1 System Behavior Under Heavy Load ($\lambda > 6$ req/s)

4.4.2 Physical Saturation vs. Deadline Exceeded Failures

4.5 Sensitivity Analysis: Deadline Relaxation (ΔD)

Chapter 5

Comparative Benchmark

5.1 Experimental Setup and Baseline Algorithms

5.2 Comparative Performance Evaluation

5.3 Trade-Off Analysis

Chapter 6

Conclusions and Future Work

6.1 Summary of Findings

6.2 Limitations of the Centralized Approach

6.3 Future Research Directions

Bibliography

- [1] BOYD, S. AND VANDENBERGHE, L. *Convex Optimization*. Cambridge University Press, Cambridge, UK (2004).
- [2] CASALICCHIO, E. ET AL. Handling of energy- and time-constrained heterogeneous satellite computing services. In *Proceedings of the IEEE International Conference on Distributed Computing Systems (ICDCS)*, pp. 1–12. IEEE (2025).
- [3] DENBY, B. AND LUCIA, B. Orbital edge computing: Nanosatellite clusters as accelerators for earth observation. *IEEE Micro*, **40** (2019), 13.
- [4] FAN, W., MENG, Q., WANG, G., BIAN, H., AND LIU, Y. Satellite edge intelligence: Drl-based resource management for task inference in leo-based satellite-ground collaborative networks. *IEEE Transactions on Mobile Computing*, **24** (2025), 10710.
- [5] GUROBI OPTIMIZATION, LLC. *Gurobi Optimizer Reference Manual* (2024). Available from: <https://www.gurobi.com>.
- [6] NEUMANN, A., AICHELE, C., LINDNER, M., AND FUX, S. Better approach to mobile ad-hoc networking (b.a.t.m.a.n.). Tech. rep., IETF Draft (2008). Draft-wunderlich-openmesh-manet-routing-00.
- [7] SCHERFKE, S. AND LÜNSDORF, O. *SimPy: Discrete Event Simulation for Python* (2023). Version 4.1.1. Available from: <https://simpy.readthedocs.io/>.
- [8] TANG, X. AND YI, X. A general metric of load balancing in δ -range. In *Advanced Parallel Processing Technologies*, pp. 311–321. Springer (2003).
- [9] WANG, S., ZHANG, Q., ZHAO, C., ET AL. Satellite computing: Vision and challenges – tiansuan constellation. *IEEE Internet of Things Journal*, **10** (2023), 17550.
- [10] WOLSEY, L. A. *Integer Programming*. John Wiley & Sons, New York, NY, USA (1998).
- [11] ZHANG, Y., LI, X., ZHOU, H., AND CHEN, D. A comprehensive survey on orbital edge computing: Systems, applications, and algorithms. *IEEE Communications Surveys & Tutorials*, **25** (2023), 2100.

-
- [12] ZHONG, L., LI, Y., GE, M.-F., FENG, M., AND MAO, S. Joint task offloading and resource allocation for leo satellite-based mobile edge computing systems with heterogeneous task demands. *IEEE Transactions on Vehicular Technology*, **74** (2025), 11337.